



Guía de Ingeniería: Consistencia Distribuida y Gestión de Catálogo

Para que nuestro ecosistema pase al siguiente nivel, implementaremos una **reacción en cadena**. El objetivo es que la información fluya de manera inteligente: cuando el **Administrador** da de alta un producto, la **Bodega** se entera automáticamente y las **Ventas** se validan con precisión quirúrgica.

A continuación, presentamos el mapa de ruta para las modificaciones, asegurando que cada microservicio mantenga su "única fuente de verdad" mientras colabora con los demás.

1. ms-stock (El Guardián):

- **Service:** Cambiar la lógica de `tieneStockSuficiente` e implementar `inicializarStock`.
- **Controller:** Exponer los nuevos endpoints de validación y creación inicial.

2. ms-pedidos (El Vendedor):

- **Feign Client:** Actualizar el contrato para enviar la cantidad solicitada.
- **Service:** Inyectar la validación cuantitativa y la orden de descuento.

3. ms-administrador (El Gestor - NUEVO):

- **Estructura:** Crear proyecto (Puerto 8083), paquetes y dependencias (Web, JPA, MySQL, Eureka, Feign).
- **Lógica:** Implementar la validación cruzada para no duplicar IDs y dar la orden de inicialización a Stock.

4. api-gateway (El Portero):

- **Config:** Agregar la ruta para el nuevo microservicio.
-

Solución Paso a Paso y Documentada

PASO 1: Reforzar la Bodega (ms-stock)

Objetivo: Que el stock sepa validar cantidades exactas y permita ser inicializado por el administrador.

En StockService.java:

```
Java
// Valida si la cantidad en DB cubre el pedido
public boolean tieneStockSuficiente(Long productoId, Integer
cantidadRequerida) {
    return stockRepository.findByProductoId(productoId)
        .map(stock -> stock.getCantidad() >= cantidadRequerida)
        .orElse(false);
}

// Crea un registro en 0 si el ID no existe (Defensa en profundidad)
@Transactional
public void inicializarStock(Long productoId) {
    if (!stockRepository.findByProductoId(productoId).isPresent()) {
        Stock nuevo = new Stock();
        nuevo.setProductoId(productoId);
        nuevo.setCantidad(0);
        stockRepository.save(nuevo);
    }
}
```

En StockController.java:

```
Java

@GetMapping("/validar/{productoId}")
public ResponseEntity<Boolean> validar(@PathVariable Long productoId,
@RequestParam Integer cantidad) {
    return ResponseEntity.ok(stockService.tieneStockSuficiente(productoId,
cantidad));
}

@PostMapping("/inicializar/{productoId}")
public ResponseEntity<Void> inicializar(@PathVariable Long productoId) {
    stockService.inicializarStock(productoId);
    return ResponseEntity.status(HttpStatus.CREATED).build();
}
```

PASO 2: Sincronizar el Cliente de Ventas (ms-pedidos)

Objetivo: Que el pedido envíe la cantidad al validar y descuento automáticamente al finalizar.

En StockClient.java (Feign):

Java

```
@FeignClient(name = "ms-stock")
public interface StockClient {
    @GetMapping("/api/stock/validar/{productoId}")
    boolean verificarStock(@PathVariable("productoId") Long productoId,
        @RequestParam("cantidad") Integer cantidad);

    @PutMapping("/api/stock/descontar/{productoId}")
    void descontarStock(@PathVariable("productoId") Long productoId,
        @RequestParam("cantidad") Integer cantidad);
}
```

En PedidoService.java:

Java

```
public Pedido crearPedido(Pedido pedido) {
    // 1. Validamos cantidad exacta vía Feign
    if (!stockClient.verificarStock(pedido.getProductoId(),
        pedido.getCantidad())) {
        throw new RuntimeException("No se puede crear el pedido: cantidad
        pedida supera el stock actual.");
    }
    // 2. Si hay stock, descontamos inmediatamente
    stockClient.descontarStock(pedido.getProductoId(), pedido.getCantidad());

    pedido.setEstado("PROCESADO");
    return pedidoRepository.save(pedido);
}
```

PASO 3: Crear el Microservicio de Administración (ms-administrador)

Objetivo: Centralizar el ingreso de productos y asegurar que nazcan con un registro en la bodega.

1. **Dependencias:** (Ya conocidas: Web, JPA, MySQL, Eureka Client, OpenFeign).
2. **Puerto:** 8083.
3. **Base de Datos:** db_productos (Puerto 3307).

Lógica del Servicio (ProductoService.java):

Java

```
@Transactional
public Producto registrarProducto(Producto producto) {
    // Comunicamos con Stock para ver si el ID ya está ocupado
    if (stockClient.verificarExistencia(producto.getId())) {
        throw new RuntimeException("El ID ya existe en el sistema de
inventario.");
    }
    Producto guardado = productoRepository.save(producto);
    // IMPORTANTE: Aquí se gatilla la creación automática en la base de datos
de Stock
    stockClient.inicializarStock(guardado.getId());
    return guardado;
}
```

PASO 4: Actualizar el Gateway (api-gateway)

Agregar en el application.yml:

YAML

```
- id: ms-administrador
  uri: lb://ms-administrador
  predicates:
    - Path=/api/productos/**
```

Secuencia de Operación y Prueba de Fuego

1. Lanzamiento: Eureka → Stock → Pedidos → Administrador → Gateway.
2. El Experimento (Postman):
 - Acción: Registrar un nuevo producto.
 - Método: POST
 - URL: `http://localhost:8080/api/productos`
 - JSON:

JSON

```
{  
  "id": 800,  
  "nombre": "Teclado Mecánico RGB",  
  "descripcion": "Switch Blue, Layout Español",  
  "precioUnitario": 55.90,  
  "categoria": "Periféricos"  
}
```

La Magia detrás de cámaras: Al recibir este JSON, el **ms-administrador** guarda los datos en `db_productos` y, en milisegundos, le envía una señal al **ms-stock**. Si revisas tu base de datos de stock, verás que el ID 800 ya existe con cantidad 0, listo para recibir suministros y ser vendido.